

INTRODUCTION

Maintaining a healthy lifestyle has become an important concern in today's busy life. Many people spend most of their day sitting while studying or working and often do not know how much physical activity they actually perform. Without proper information about daily steps, calories burned and exercise time, it becomes difficult for a person to set fitness goals and follow them regularly.

In many situations, users may not remember how long they exercised, how many calories they burned or whether they reached their daily activity target. Writing this information manually is inconvenient and is often forgotten. Therefore, a mobile application that automatically records daily activity and presents it in a simple visual form can help users stay active and motivated.

FitTracker – Fitness and Workout Tracking Application is an Android-based mobile application developed to provide a simple and convenient platform for daily step counting, workout tracking and fitness progress monitoring. The main purpose of the application is to help users understand their daily physical activity and encourage them to reach their personal fitness goals.

The application counts steps automatically in the background, shows Heart Points, steps, calories, distance and move minutes on a home dashboard with activity rings and weekly charts, and allows users to track live workouts using a stopwatch. Users can also add past activities manually, view their workout history in a journal and maintain a profile with BMI calculation and personal daily goals.

The application is developed using **Java** for application logic, **XML** for user interface design and **SQLite** for local database management. Android Studio and the Android SDK are used as the development environment. Android Sensor Framework, Foreground Services and Notifications are used for background step counting.

Background of the Project

Physical inactivity is one of the major causes of many lifestyle-related health problems. Health experts recommend walking a certain number of steps every day and doing regular exercise. However, most people do not have a simple method to measure their daily activity and compare it with their goals.

Traditional methods of tracking fitness involve writing workout details in a notebook or using separate devices such as pedometers and fitness bands. These methods can be costly or inconvenient, and the recorded information is often incomplete. Since almost every smartphone today contains motion sensors, a mobile application can use these sensors to record daily activity automatically.

FitTracker provides a centralized solution where users can view their daily steps, Heart Points, calories and workouts through a single Android application. The system is designed with a simple and user-friendly interface, similar to popular fitness applications, so that users can easily understand their progress and stay motivated.

Problem Statement

Users may face difficulty in monitoring their daily physical activity and keeping a record of their workouts efficiently.

The existing manual fitness tracking process may have the following difficulties:

1. Users may not know how many steps they walk in a day.
2. Calories burned and distance covered are difficult to calculate manually.
3. Workout details may be maintained manually or not recorded at all.
4. Users may not have a convenient method to set and monitor daily fitness goals.
5. Separate fitness devices such as pedometers and fitness bands can be costly.
6. There may be no simple method to view weekly progress and activity history.
7. Fitness information may not be organized in a centralized system.
8. Lack of regular feedback can reduce the motivation to stay active.

Therefore, a mobile application is proposed to provide centralized and automatic fitness tracking and workout management facilities.

OBJECTIVES

Main Objective

The main objective of FitTracker is to develop an Android-based fitness tracking application that allows users to count daily steps, track workouts and monitor their fitness progress conveniently while storing all activity information digitally on the device.

Specific Objectives

The objectives of the project are:

1. To develop a user-friendly Android application for fitness tracking.
2. To count the user's daily steps automatically using the step counter sensor.
3. To continue step counting in the background using a foreground service.
4. To calculate calories burned, distance covered and move minutes.
5. To award Heart Points based on moderate and vigorous activities.
6. To allow users to track live workouts using a stopwatch.
7. To allow users to add past activities manually.
8. To maintain workout history digitally in a journal.
9. To display daily progress using activity rings and weekly bar charts.
10. To allow users to set personal daily step and Heart Point goals.
11. To calculate the Body Mass Index (BMI) of the user.
12. To use SQLite for storing and managing application data locally.
13. To provide a simple and interactive mobile interface using Java and XML.
14. To show daily step progress through an ongoing notification.

SCOPE OF THE PROJECT

Scope

The scope of FitTracker is to provide a mobile-based solution for tracking daily physical activity, workouts and fitness goals. The application focuses on making fitness tracking easier and more motivating for users.

The major areas covered by the application are:

3.1 Activity Tracking

The application allows users to:

- Count daily steps automatically.
- View Heart Points and steps on activity rings.
- View calories burned, distance and move minutes.
- View 7-day bar charts for steps and Heart Points.
- View the weekly goal status and activity streak.
- Receive an ongoing notification showing today's steps.

3.2 Workout Management

The application provides users with workout tracking functions.

Users can:

- Select an activity such as Walking, Running, Cycling, Yoga or HIIT.
- Start, pause, resume and finish a live workout.
- View live calories, Heart Points, steps and distance.
- Add a past activity manually with duration and distance.
- View the complete workout history in the journal.
- Delete a workout entry from the journal.

3.3 Database Scope

SQLite is used for local data management. The database can maintain information related to:

- Workouts
- Daily step totals
- Calories and Heart Points
- Workout distance and duration
- Activity type information

3.4 Profile and Goals Scope

The application allows users to store their name, age, height and weight, set daily step and Heart Point goals and view their Body Mass Index (BMI) with its category. Profile information is stored using SharedPreferences.

3.5 Limitations

The current application is mainly designed as an academic Mobile Application Development project.

Some advanced real-world facilities may require additional development, such as:

- Cloud-based backup and synchronization.
- Heart rate measurement using wearable devices.
- GPS-based route tracking for outdoor workouts.
- Diet and water intake tracking.
- Integration with smart watches and fitness bands.
- Social sharing and fitness challenges.

These features can be considered for future enhancement.

SYSTEM DESIGN

4.1 System Overview

FitTracker is an Android-based fitness tracking application developed using Java and XML. The application mainly works with the following parts:

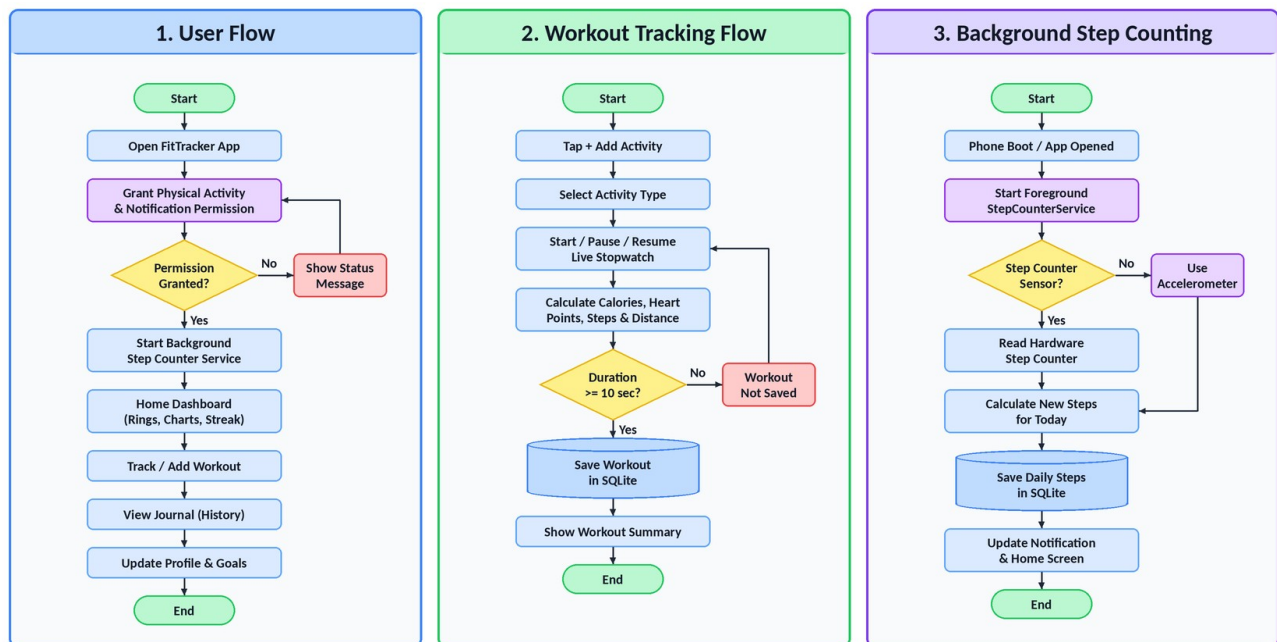
1. **Activity and Workout Tracking (User Interface)**
2. **Background Step Counting Service**

The user interacts with the application to view daily progress, track workouts, view the journal and update the profile and goals.

The background service counts steps continuously using the phone's sensors and updates the notification and home screen.

The application uses SQLite to store important application information locally.

Basic System Flow :



4.2 System Architecture

The system can be divided into the following layers:

i)Presentation Layer

It is developed using:

- XML
- Activities
- Fragments
- BottomNavigationView
- RecyclerView
- Material Design 3 components (Chips, Cards, Buttons, Bottom Sheets)
- Custom Views (RingView and BarChartView drawn on Canvas)

ii)Application Logic Layer

This layer contains the Java code responsible for:

- Screen navigation.
- Workout tracking and stopwatch.
- Manual activity entry.
- Calories, Heart Points and distance calculation.
- BMI calculation.
- Goal and streak calculation.
- Data validation.

iii) Database Layer

SQLite and SharedPreferences are used for storing application data.

The database manages information related to:

- Workouts
- Daily steps
- User profile
- Daily goals

iv) Sensor and Service Layer

Android Sensor Framework (Step Counter and Accelerometer), Foreground Service, Boot Receiver and Notifications support automatic background step counting.

4.3 Main Components

The major components of FitTracker are:

1.Home Module

Displays activity rings, daily statistics, weekly charts and recent activities.

2.Workout Tracking Module

Tracks live workouts using a stopwatch with live calories and Heart Points.

3.Manual Entry Module

Allows users to add past activities with type, duration and distance.

4.Journal Module

Displays the complete workout history with a 7-day summary.

5.Profile Module

Maintains user details, daily goals and BMI information.

6.Step Counter Module

Counts steps in the background using sensors and a foreground service.

7.Database Module

Stores and retrieves application information using SQLite.

4.4 Database Design

SQLite is used as the local database management system.

The major logical entities include:

i)Workouts

Stores workout information.

Information includes:

- Workout ID
- Activity type
- Start time
- Duration
- Distance

- Calories
- Heart Points
- Steps

ii)Daily Steps

Stores the total steps for each day.

Information includes:

- Day (date)
- Total steps

iii)User Profile (SharedPreferences)

Stores user profile and goal information.

Information includes:

- Name
- Age
- Height
- Weight
- Daily step goal
- Daily Heart Point goal

iv)Step Tracker State (SharedPreferences)

Stores the current state of step counting.

Information includes:

- Current day
- Today's step count
- Last raw step counter value

CODE

- MainActivity.java

```
package com.example.fittracker;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.Fragment;

public class MainActivity extends AppCompatActivity {

    private BottomNavigationView bottomNav;
    private ExtendedFloatingActionButton fab;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        bottomNav = findViewById(R.id.bottom_nav);
        fab = findViewById(R.id.fab_add);

        bottomNav.setOnItemSelectedListener(item -> {
            showScreen(item.getItemId()); return true; });
        fab.setOnClickListener(v -> showAddOptions());

        if (savedInstanceState == null) showScreen(R.id.nav_home);
        requestPermissions();
    }

    @Override
    protected void onResume() {
        super.onResume();
        // Make sure background counting is running
        StepCounterService.start(this);
    }

    private void showScreen(int itemId) {
        Fragment fragment;
        if (itemId == R.id.nav_journal) fragment = new JournalFragment();
        else if (itemId == R.id.nav_profile) fragment = new ProfileFragment();
        else fragment = new HomeFragment();
        getSupportFragmentManager().beginTransaction()
            .replace(R.id.fragment_container, fragment)
            .commit();
        if (itemId == R.id.nav_profile) fab.hide();
        else fab.show();
    }
}
```

- **WorkoutActivity.java**

```
package com.example.fittracker;

/** Live workout tracking: stopwatch, calories, Heart Points and steps. */
public class WorkoutActivity extends AppCompatActivity
    implements StepTracker.Listener {

    private static final long MIN_WORKOUT_SEC = 10;
    private ActivityType type = ActivityType.WALKING;
    private boolean started, running;
    private long accumulatedMs, segmentStart, startWallTime;
    private int workoutSteps;

    private void toggle() {
        if (!started) {
            started = true;
            startWallTime = System.currentTimeMillis();
        }
        if (running) {
            accumulatedMs += SystemClock.elapsedRealtime() - segmentStart;
            running = false;
            handler.removeCallbacks(ticker);
            btnStart.setText("Resume");
        } else {
            running = true;
            segmentStart = SystemClock.elapsedRealtime();
            handler.post(ticker);
            btnStart.setText("Pause");
        }
        updateStats();
    }

    private void updateStats() {
        long sec = elapsedSec();
        tvTimer.setText(DateUtil.timer(sec));
        tvCalories.setText(String.valueOf(Math.round(
            FitCalc.workoutCalories(type, prefs.getWeightKg(), sec))));
        tvHeart.setText(String.valueOf(FitCalc.heartPoints(type, sec)));
        tvSteps.setText(String.valueOf(workoutSteps));
    }

    private void finishWorkout() {
        if (running) toggle();
        long sec = elapsedSec();
        if (sec < MIN_WORKOUT_SEC) return; // too short to save
        Workout w = Workout.create(type, startWallTime, sec,
            distanceMeters(), workoutSteps, prefs.getWeightKg());
        DatabaseHelper.get(this).insertWorkout(w);
    }
}
```

- **HomeFragment.java**

```
package com.example.fittracker.ui;

public class HomeFragment extends Fragment
    implements StepTracker.Listener, Refreshable {

    private RingView ring;
    private TextView tvGreeting, tvCalories, tvDistance, tvMoveMin;
    private BarChartView chart;

    @Override
    public void onResume() {
        super.onResume();
        // Steps are counted by StepCounterService; this screen only listens
        StepTracker.addObserver(this);
        load(true);
    }

    @Override
    public void onPause() {
        super.onPause();
        StepTracker.removeObserver(this);
    }

    @Override
    public void onStepsUpdated(int todaySteps) {
        if (isResumed()) load(false);
    }

    private void load(boolean animate) {
        UserPrefs prefs = new UserPrefs(requireContext());
        DayStats[] days = DayStats.lastDays(requireContext(), 7);
        DayStats today = days[days.length - 1];

        String firstName = prefs.getName().trim().split(" ")[0];
        tvGreeting.setText(greeting() + ", " + firstName);

        ring.setData(today.heartPoints, prefs.getHeartGoal(),
            today.steps, prefs.getStepGoal(), animate);
        tvCalories.setText(String.valueOf(Math.round(today.calories)));
        tvDistance.setText(String.format("%.2f", today.distanceKm));
        tvMoveMin.setText(String.valueOf(today.moveMinutes));
        showChart(animate);
    }

    private static String greeting() {
        int hour = Calendar.getInstance().get(Calendar.HOUR_OF_DAY);
        if (hour < 12) return "Good morning";
        if (hour < 17) return "Good afternoon";
        return "Good evening";
    }
}
```

- **ProfileFragment.java**

```
package com.example.fittracker.ui;
```

```
public class ProfileFragment extends Fragment {

    private EditText etName, etAge, etHeight, etWeight;
    private EditText etStepGoal, etHeartGoal;
    private TextView tvBmi, tvBmiCategory;
    private UserPrefs prefs;

    private void save() {
        String name = etName.getText().toString().trim();
        int age = parseInt(etAge);
        float height = parseFloat(etHeight);
        float weight = parseFloat(etWeight);
        int stepGoal = parseInt(etStepGoal);
        int heartGoal = parseInt(etHeartGoal);

        if (TextUtils.isEmpty(name)) {
            etName.setError("Enter your name");
            return;
        }
        if (age < 5 || age > 120) {
            etAge.setError("Enter a valid age");
            return;
        }
        if (height < 50 || height > 250) {
            etHeight.setError("Height in cm (50-250)");
            return;
        }
        if (weight < 20 || weight > 300) {
            etWeight.setError("Weight in kg (20-300)");
            return;
        }
        prefs.save(name, age, height, weight, stepGoal, heartGoal);
        updateHeader();
        Toast.makeText(requireContext(), "Profile saved",
            Toast.LENGTH_SHORT).show();
    }

    private void updateHeader() {
        double bmi = FitCalc.bmi(prefs.getWeightKg(), prefs.getHeightCm());
        tvBmi.setText(String.format("%.1f", bmi));
        tvBmiCategory.setText(FitCalc.bmiCategory(bmi));
    }
}
```

- **DatabaseHelper.java**

```
package com.example.fittracker.data;

/** SQLite storage for workouts and daily step totals. */
public class DatabaseHelper extends SQLiteOpenHelper {

    private static final String DB_NAME = "fittracker.db";
    private static final int DB_VERSION = 1;

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE workouts ("
            + "id INTEGER PRIMARY KEY AUTOINCREMENT, "
            + "type TEXT NOT NULL, "
            + "start_time INTEGER NOT NULL, "
            + "duration_sec INTEGER NOT NULL, "
            + "distance_m REAL DEFAULT 0, "
            + "calories REAL DEFAULT 0, "
            + "heart_points INTEGER DEFAULT 0, "
            + "steps INTEGER DEFAULT 0)");
        db.execSQL("CREATE TABLE daily_steps ("
            + "day TEXT PRIMARY KEY, "
            + "steps INTEGER NOT NULL)");
    }

    public long insertWorkout(Workout w) {
        ContentValues cv = new ContentValues();
        cv.put("type", w.type.name());
        cv.put("start_time", w.startTime);
        cv.put("duration_sec", w.durationSec);
        cv.put("distance_m", w.distanceM);
        cv.put("calories", w.calories);
        cv.put("heart_points", w.heartPoints);
        cv.put("steps", w.steps);
        w.id = getWritableDatabase().insert("workouts", null, cv);
        return w.id;
    }

    public void saveSteps(String day, int steps) {
        ContentValues cv = new ContentValues();
        cv.put("day", day);
        cv.put("steps", steps);
        getWritableDatabase().insertWithOnConflict("daily_steps",
            null, cv, SQLiteDatabase.CONFLICT_REPLACE);
    }
}
```

- **StepTracker.java**

```
package com.example.fittracker.sensor;

/** Counts today's steps using the step counter or accelerometer. */
public class StepTracker implements SensorEventListener {

    public void start() {
        if (stepCounter != null && hasPermission(appContext)) {
            sensorManager.registerListener(this, stepCounter,
                SensorManager.SENSOR_DELAY_NORMAL);
        } else if (accelerometer != null) {
            sensorManager.registerListener(this, accelerometer,
                SensorManager.SENSOR_DELAY_GAME);
        }
    }

    @Override
    public void onSensorChanged(SensorEvent event) {
        if (event.sensor.getType() == Sensor.TYPE_STEP_COUNTER) {
            // Value is the total since reboot; add the difference
            int raw = (int) event.values[0];
            int last = sp.getInt(K_LAST_RAW, -1);
            int delta;
            if (last < 0) delta = 0;           // first reading
            else if (raw >= last) delta = raw - last;
            else delta = raw;                 // device rebooted
            sp.edit().putInt(K_LAST_RAW, raw).apply();
            if (delta > 0) addSteps(delta);
        } else if (event.sensor.getType() == Sensor.TYPE_ACCELEROMETER) {
            float x = event.values[0], y = event.values[1];
            float z = event.values[2];
            double magnitude = Math.sqrt(x * x + y * y + z * z);
            gravity = 0.9 * gravity + 0.1 * magnitude; // low-pass filter
            double dynamic = magnitude - gravity;
            long now = System.currentTimeMillis();
            if (!aboveThreshold && dynamic > 1.8 && now - lastStepMs > 300) {
                aboveThreshold = true;
                lastStepMs = now;
                addSteps(1);
            } else if (aboveThreshold && dynamic < 0.5) {
                aboveThreshold = false;
            }
        }
    }
}
```

- **StepCounterService.java**

```
package com.example.fittracker.sensor;

/** Foreground service that counts steps even when the app is closed. */
public class StepCounterService extends Service
    implements StepTracker.Listener {

    private static final String CHANNEL_ID = "step_counter";
    private static final int NOTIFICATION_ID = 1;
    private StepTracker tracker;

    public static void start(Context context) {
        if (!StepTracker.hasPermission(context)) return;
        ContextCompat.startForegroundService(context,
            new Intent(context, StepCounterService.class));
    }

    @Override
    public void onCreate() {
        super.onCreate();
        createChannel();
        tracker = new StepTracker(this, this);
        ServiceCompat.startForeground(this, NOTIFICATION_ID,
            buildNotification(tracker.getTodaySteps()),
            ServiceInfo.FOREGROUND_SERVICE_TYPE_HEALTH);
        tracker.start();
    }

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        return START_STICKY;
    }

    private Notification buildNotification(int steps) {
        int goal = new UserPrefs(this).getStepGoal();
        return new NotificationCompat.Builder(this, CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_walk)
            .setContentTitle(steps + " steps today")
            .setProgress(goal, Math.min(steps, goal), false)
            .setOngoing(true)
            .setSilent(true)
            .build();
    }

    @Override
    public void onDestroy() {
        if (tracker != null) tracker.stop();
        super.onDestroy();
    }
}
```

- **FitCalc.java**

```
package com.example.fittracker.util;

import com.example.fittracker.model.ActivityType;

/** Formulas used to estimate fitness metrics. */
public final class FitCalc {

    /** Calories = MET x weight (kg) x hours. */
    public static double workoutCalories(ActivityType type,
                                         double weightKg, long seconds) {
        return type.met * weightKg * seconds / 3600.0;
    }

    public static int heartPoints(ActivityType type, long seconds) {
        return (int) (seconds / 60) * type.heartPointsPerMinute;
    }

    /** Average stride length is roughly 41.5% of body height. */
    public static double strideMeters(double heightCm) {
        return heightCm * 0.415 / 100.0;
    }

    public static double stepsToMeters(int steps, double heightCm) {
        return steps * strideMeters(heightCm);
    }

    /** About 0.04 kcal per step for a 70 kg person. */
    public static double stepCalories(int steps, double weightKg) {
        return steps * 0.04 * (weightKg / 70.0);
    }

    public static double bmi(double weightKg, double heightCm) {
        double m = heightCm / 100.0;
        return m <= 0 ? 0 : weightKg / (m * m);
    }

    public static String bmiCategory(double bmi) {
        if (bmi <= 0) return "-";
        if (bmi < 18.5) return "Underweight";
        if (bmi < 25) return "Healthy weight";
        if (bmi < 30) return "Overweight";
        return "Obese";
    }
}
```

- **activity_main.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/app_bg">

    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottom_nav"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_alignParentBottom="true"
        android:background="@color/card_bg"
        app:labelVisibilityMode="labeled"
        app:menu="@menu/bottom_nav" />

    <FrameLayout
        android:id="@+id/fragment_container"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_above="@id/bottom_nav" />

    <com.google.android.material.floatingactionbutton
        .ExtendedFloatingActionButton
        android:id="@+id/fab_add"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_above="@id/bottom_nav"
        android:layout_alignParentEnd="true"
        android:layout_margin="18dp"
        android:text="Add activity"
        android:textColor="@color/white"
        app:backgroundTint="@color/primary"
        app:icon="@drawable/ic_add" />
</RelativeLayout>
```

- **AndroidManifest.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission
        android:name="android.permission.ACTIVITY_RECOGNITION" />
    <uses-permission
        android:name="android.permission.FOREGROUND_SERVICE" />
    <uses-permission
        android:name="android.permission.FOREGROUND_SERVICE_HEALTH" />
    <uses-permission
        android:name="android.permission.POST_NOTIFICATIONS" />
    <uses-permission
        android:name="android.permission.RECEIVE_BOOT_COMPLETED" />

    <application
        android:allowBackup="true"
        android:label="@string/app_name"
        android:theme="@style/Theme.FitTracker">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category
                    android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity android:name=".WorkoutActivity"
            android:exported="false" />

        <service android:name=".sensor.StepCounterService"
            android:exported="false"
            android:foregroundServiceType="health" />

        <receiver android:name=".sensor.BootReceiver"
            android:exported="true">
            <intent-filter>
                <action
                    android:name="android.intent.action.BOOT_COMPLETED" />
            </intent-filter>
        </receiver>
    </application>
</manifest>
```

OUTPUT:

[Paste screenshot here]

Home Screen

Home Screen

[Paste screenshot here]

Track Workout Screen

Track Workout Screen

[Paste screenshot here]

Add Activity Screen

[Paste screenshot here]

Journal Screen

Add Activity Screen

Journal Screen

[Paste screenshot here]

Profile Screen

[Paste screenshot here]

Step Counter Notification

Profile Screen

Step Counter Notification

ADVANTAGES

1. **Automatic Step Counting**
Steps are counted automatically without any manual effort.
2. **Background Tracking**
Steps are counted even when the application is closed.
3. **Live Workout Tracking**
Users can track workouts with a live stopwatch and statistics.
4. **Fitness Metrics**
Calories, distance, move minutes and Heart Points are calculated automatically.
5. **Goal Setting**
Users can set personal daily step and Heart Point goals.
6. **Visual Progress**
Activity rings and weekly bar charts show progress clearly.
7. **Workout History**
All workouts are maintained systematically in the journal.
8. **BMI Calculation**
The application calculates BMI and shows the BMI category.
9. **User-Friendly Interface**
The application provides a simple and attractive mobile interface.
10. **Local Database Support**
SQLite provides local storage for application data.
11. **No Extra Device Required**
The application uses the phone's own sensors instead of a fitness band.
12. **Android-Based Solution**
Users can access the system through an Android mobile application.

FUTURE SCOPE :

1. Cloud Backup
The local SQLite database can be extended with a cloud-based database so that user data can be backed up and synchronized across devices.
2. GPS Route Tracking
GPS can be used to record the route, speed and exact distance of outdoor workouts such as running and cycling.
3. Wearable Integration
The application can be connected with smart watches and fitness bands to read heart rate and other health data.
4. Diet and Water Tracking
Users can be provided with features to record meals, calorie intake and daily water consumption.
5. Sleep Tracking
Sleep duration and sleep quality can be monitored to provide a complete health view.
6. Social Challenges
Users can compete with friends in step challenges and share their achievements.
7. Advanced Reports
Monthly and yearly reports can be generated to show long-term fitness progress.
8. Notifications
Push notifications can be added for:
 - Workout reminders.
 - Goal achievement.
 - Inactivity alerts.
 - Weekly progress summary.
9. Personal Analytics
Users can be provided with analytical information such as:
 - Average daily steps.
 - Most frequent activities.
 - Best performance days.
 - Daily and monthly activity trends.

ACHIEVED COURSE OUTCOMES:

- CO1: Describe features of Android operating system.
- CO2: Setup and configure the Android development environment.
- CO3: Design user-friendly graphical interfaces
- CO4 : Build Android applications using standard UI components.
- CO5: Develop Android applications with integrated database functionality
- CO6: Deploy Android application.

CONCLUSION

The FitTracker application successfully provides a simple and organized solution for tracking daily physical activity and workouts through a mobile application. The main purpose of the project is to help users know how active they are every day and to motivate them to reach their fitness goals.

The application is developed using Android Studio with Java and XML, along with SQLite for storing and managing application data. The use of these technologies helped in implementing important mobile application development concepts such as activities and fragments, screen navigation, user interface design, sensors, foreground services, notifications, runtime permissions, database connectivity and custom views.

FitTracker counts steps automatically in the background, calculates calories, distance and Heart Points, and allows users to track live workouts, add past activities, view their history and manage their profile and goals. Activity rings and weekly charts make the progress easy to understand.

The project also helped in understanding how different components of an Android application work together. The implementation of SQLite provided practical knowledge of local data storage and database operations, while Java was used to handle application logic, sensor data and user interactions. XML was used to design the application interfaces and provide a structured and user-friendly experience.

Overall, FitTracker demonstrates how mobile application development concepts can be applied to solve a real-world problem. The project provides a foundation that can be further improved with additional features and technologies in the future.

Thus, the FitTracker project successfully meets its primary objective of developing a mobile-based fitness and workout tracking application while providing practical knowledge and experience in Android application development, Java programming, XML-based UI design, SQLite database management, and application integration.

REFERENCES

1. Android Developers, **Android Developer Documentation**, Google.
2. Android Developers, **Activities and Activity Lifecycle**, Google.
3. Android Developers, **Fragments**, Google.
4. Android Developers, **SQLite Database in Android**, Google.
5. Android Developers, **“Sensors Overview and Motion Sensors,”** Google.
6. Android Developers, **“Foreground Services,”** Google.
7. Android Developers, **Notifications and Runtime Permissions**, Google.
8. Oracle, **Java Documentation**, Oracle Corporation.
9. Material Design, **Material Components for Android**, Google.
10. Android Studio, **Integrated Development Environment for Android Application Development**, Google.